

Robotic Motion — Operating Guide

Waveshare “Servo Driver with ESP32” + Feitech STS3215 6-DOF arm

Scripts & logs: [/home/user/robotic_motion/](#) • Generated 2026-07-25

1. What the arm is and how it is controlled

- **Board:** Waveshare “Servo Driver with ESP32”, a Silicon Labs CP2102 USB-UART bridge on `/dev/ttyUSB0`.
- **Servos:** Feitech **STS3215** bus servos, IDs 1–6 (ST family: absolute position 0–4095 = 360°, ~11.38 counts/deg). Joint 5 = wrist bend, Joint 6 = end-flange twist.
- **Power:** the servos need their own DC motor supply ON. Without it the board boots but every servo is dead (this was the original problem).
- **Control path:** in normal mode the USB port is **debug-only** — the firmware ignores USB commands. To command servos you enable **SERIAL_FORWARDING**, which turns the USB port into a transparent bridge to the 1 Mbps servo bus. Then the PC speaks the Feitech protocol directly.
- **Enabling forwarding** is done from the board’s own WiFi web page (access point **ESP32_DEV**, password **12345678**, <http://192.168.4.1>).

THE GOLDEN RULE (important quirk)

Every time a program opens `/dev/ttyUSB0`, the ESP32 **reboots** (its reset pin is wired to the USB control lines) and **SERIAL_FORWARDING** switches **OFF**. So the order is always: **start the script first**, and only **after** it prints “PORT OPEN ... waiting” do you (re)enable forwarding on the phone. Each script run needs one enable.

2. Every-session checklist

- Motor DC power supply **ON**; USB cable from the board to the PC.
- The board’s small OLED is lit.
- Run all scripts with this Python (it has pyserial): `/home/user/robotic_arm/venv/bin/python`
- You are in the **dialout** group (already done) so `/dev/ttyUSB0` is usable.

3. Enable SERIAL_FORWARDING (phone) — do this when a script says “waiting”

- 1 On the phone WiFi settings, connect to **ESP32_DEV**, password **12345678**. Ignore the “no internet” warning and stay on it (turn mobile data off so the browser uses the board).
- 2 Open a browser to <http://192.168.4.1> (http, not https). If it was already open, **reload** the page.
- 3 Tap “**Start Serial Forwarding**” and tap **OK** on the confirm dialog.
- 4 Confirm it worked: the OLED shows **SERIAL FORWARDING** and the button relabels to “**Stop Serial Forwarding**”.

You do **not** need “Start Searching” or to pick a servo — the scripts address joints by ID directly. **Never** tap “Set Middle Position” — it rewrites a servo’s zero.

4. Move joints by a fixed angle — `scs_move.py`

A safe, one-shot move: it read-checks framing (commands the joint to its current spot = no motion), does a slow 5° probe, then the full angle, then returns to the start position. One joint at a time, torque stays on.

Run it (example: joints 5 and 6, 20°):

```
/home/user/robotic_arm/venv/bin/python /home/user/robotic_motion/scs_move.py
--family ST --joints 5 6 --deg 20 --go
```

- 1 Run the command above. It opens the port (board reboots) and prints “PORT OPEN ... waiting”.
- 2 Do Section 3 (enable forwarding). Within a couple of seconds it prints “Bridge LIVE” and moves the joints.
- 3 Watch the arm; the script prints start / probe / moved / returned angles and exits.

Options:

- **--joints** list of servo IDs, e.g. **--joints 5** or **--joints 4 5 6**.
- **--deg** angle in degrees (default 20). It always moves **toward the centre** of travel (safest).
- **Dry run:** omit **--go** to print the plan and move nothing.

5. Keep the joints moving (continuous) — `scs_oscillate.py`

Sweeps each joint as a slow sine around its current position for a set time, then returns to start. Writes a timestamped log to `/home/user/robotic_motion/logs/`.

Run it (example: joints 4, 5, 6 for 30 s):

```
/home/user/robotic_arm/venv/bin/python /home/user/robotic_motion/scs_oscillate.py
--secs 30 --amp-deg 15 --freq 0.3 --joints 4 5 6 --go
```

- 1 Run the command; when it prints “PORT OPEN ... waiting”, do Section 3 to enable forwarding.
- 2 The joints weave for the chosen duration (each logged ~once/second: target vs actual counts), then ease back to their start positions.

Options:

- **--secs** duration seconds (default 30).
- **--amp-deg** swing amplitude ±degrees (default 15). Keep it modest; targets are clamped to a safe count window.
- **--freq** oscillation frequency in Hz (default 0.3 = one full back-and-forth every ~3.3 s).
- **--joints** which servo IDs to move together.

6. Logs

- Location: `/home/user/robotic_motion/logs/oscillate_YYYYMMDD_HHMMSS.log`.
- Contents: run parameters, start positions, framing check, per-second target-vs-actual counts/degrees, and the final returned positions.

- The move script (scs_move.py) prints its results to the terminal; redirect with **> run.log 2>&1** if you want it saved too.

7. Running the R_sine sinusoidal motion on the real arm

What R_sine is: the simulation project (package **src/Rsine**) that solves 6-DOF inverse kinematics to trace a sine wave on a configurable plane and plays it in RViz by publishing /joint_states. On hardware we must convert those joint angles into STS3215 servo counts and stream them to the bus.

The one missing piece = a per-joint calibration map. R_sine angles are in URDF radians; each servo has its own absolute zero and rotation direction. For every joint we need: $\text{count} = \text{zero}_i + \text{dir}_i \times \text{theta_rad} \times (4096 / 2\pi)$.

Calibration procedure (do once):

- 1 Enable forwarding (Section 3).
- 2 Put the arm in a known reference pose (ideally the URDF “all-zeros” home you use in RViz). Read all six servo counts with: **/home/user/robotic_arm/venv/bin/python /home/user/robotic_motion/scs_diag.py** — those counts are the **zero_i** for that pose.
- 3 Find each joint’s direction: nudge one joint a known +angle with scs_move.py and see whether its count went up (dir=+1) or down (dir=-1). The scale is fixed at ~11.38 counts/deg.
- 4 Record zero_i and dir_i for joints 1–6.

Then stream the trajectory:

- 1 Generate R_sine’s joint trajectory (its IK already precomputes the waypoint angles q for the whole sine path).
- 2 Map each waypoint’s six angles to counts with the calibration, clamp to each joint’s safe range, and stream them over the bridge (a sync-write of all six servos per waypoint) at a steady rate matched to the servo speed limit.
- 3 Start slow: run the first few waypoints, verify the tip traces the sine and no joint fights a limit, then run the full path.

Status: the streaming script (scs_rsine.py) is **not built yet** — it needs the calibration numbers above. Once you capture zero_i / dir_i I can generate it. Caveats: this small arm has a much smaller reach than the sim, so scale the sine amplitude/width down; watch joint 1 (its bus cable has been intermittent); keep speeds low on the first runs.

8. Troubleshooting

- **Script stuck on “waiting”:** forwarding isn’t on. Reload 192.168.4.1 and tap Start Serial Forwarding. Reconnect the phone to ESP32_DEV if it dropped during the reboot.
- **A joint shows Type -1 / not detected (esp. joint 1):** check the motor DC power and reseal that servo’s bus cable.
- **“could not open port / busy”:** another script still holds /dev/ttyUSB0 — stop it first.
- **act=None in an oscillation log:** harmless read hiccup on the bus; the joint still moves (writes are separate from reads).
- **Board unreachable after many runs:** power-cycle the board; ESP32_DEV reappears in a few seconds.

9. Safety

- Keep hands and cables clear of the arm before any --go run.
- First run of anything new: small amplitude, low speed, one joint.
- Never tap “Set Middle Position” on the web UI (rewrites the servo zero / calibration).
- To stop: press Ctrl-C (scripts ease back to start), or cut the motor power supply for an immediate limp stop.

Quick command reference

```
# read-only check (type, IDs, positions):  
/home/user/robotic_arm/venv/bin/python /home/user/robotic_motion/scs_diag.py  
  
# move joints a fixed angle (returns to start):  
/home/user/robotic_arm/venv/bin/python /home/user/robotic_motion/scs_move.py  
--family ST --joints 5 6 --deg 20 --go  
  
# keep joints moving for 30 s:  
/home/user/robotic_arm/venv/bin/python /home/user/robotic_motion/scs_oscillate.py  
--secs 30 --amp-deg 15 --freq 0.3 --joints 4 5 6 --go
```